

User Manual: Testing and Validation Guide

Latency-Constrained Dark Pool Liquidity Routing

Tianchi He

March 14, 2026

1 Introduction

This document provides a comprehensive guide for compiling, executing, and validating the Latency-Constrained Dark Pool Liquidity Routing algorithm. The underlying mathematical model utilizes a Time-Expanded Graph (TEG) synthesized with vertex-splitting to compute the absolute maximum liquidity that can be routed through a strictly constrained financial network.

2 Input and Output Specifications

To ensure structural integrity during execution, the algorithm enforces a strict separation between systemic constants (passed via standard input) and the network topology (passed via a text file).

2.1 Standard Input (Systemic Constraints)

Upon execution, the command-line interface will prompt the user for the following strict operational bounds:

- **N (Integer):** The total number of distinct physical entities (broker-dealers, dark pools, exchanges) in the network.
- **K (Integer):** The absolute temporal bound in microseconds. Flow arriving after this timeframe is mathematically rejected.
- **Source (Integer):** The origin node ID ($0 \leq S < N$).
- **Sink (Integer):** The destination market node ID ($0 \leq T < N$).
- **Default Processing Capacity (Integer):** The regulatory throughput cap applied to all internal matching engines. A value of -1 denotes infinite capacity.

2.2 File Input (Network Topology)

The spatial connections must be supplied via a plaintext file (e.g., `network.txt`). Each line represents a directed edge and must follow the space-separated format:

`u v capacity [latency]`

- `u`: The origin physical node ID.
- `v`: The destination physical node ID.
- `capacity`: The maximum volume of shares the connection can transmit in a single burst.
- `latency` (Optional): The traversal time in microseconds. If omitted, the parser defaults strictly to 1 microsecond.

Note: Lines beginning with # are treated as comments and safely ignored.

2.3 Output Format

The execution engine is configured strictly to compute the upper bound of network throughput. The final output rendered to the console is a single definitive integer representing the absolute maximum volume of shares successfully routed to the Sink within the K microsecond bound.

```
1 === EXECUTION RESULTS ===
2 Maximum routeable liquidity within 5 microseconds: 100 shares
```

Expected Console Output

3 Execution Protocol

The algorithm requires no external dependencies beyond the standard Python 3 runtime environment.

1. Save the algorithm source code as `router.py`.
2. Define your test topology in a plaintext file, such as `topology.txt` and put in the same directory as source code.
3. Initialize the engine via the terminal:

```
1 python3 router.py
2
```

4. Follow the interactive prompts to define the system constraints and supply the path to your `topology.txt` file.

Validation Methodology

To mathematically verify the validity of the algorithm, one must construct non-trivial test cases that force the Edmonds-Karp solver to navigate competing constraints: spatial bottlenecks, temporal deadlines, resting liquidity, and the continuous nature of data flow over multiple time steps.

3.1 The Benchmark Test Case

Create a file named `benchmark.txt` containing the following asymmetrical topology:

```
1 # Source to Initial Brokers
2 0 1 60 1
3 0 2 50 2
4
5 # Inter-Dealer Routing
6 1 3 40 1
7 1 5 25 3
8 2 3 30 1
9 2 4 40 2
10
11 # Dark Pool to Market (Sink)
12 3 4 60 1
13 4 5 80 1
```

`benchmark.txt`

3.2 Executing the Benchmark

Launch the application and supply the following parameters. Ensure that the source and sink nodes have been programmatically uncapped (capacity set to -1) within the execution script to prevent self-imposed institutional bottlenecks.

- **File:** `benchmark.txt`
- **N:** 6
- **K:** 5
- **Source:** 0
- **Sink:** 5
- **Default Processing Capacity:** 40

3.3 Validating the Output

If the algorithm is topologically sound, it will output exactly **105 shares**. To manually verify this correctness, one must trace the bounds and recognize that flow is continuous across the time horizon, rather than a single initial burst:

1. **Continuous Squeezing of Node 1:** The Source continuously feeds Node 1. At $T = 1$, Node 1 processes its maximum of 40 shares. It routes 25 shares down the slow, direct cable ($1 \rightarrow 5$, latency 3), which arrive safely at $T = 4$. It routes the remaining 15 shares down the complex path ($1 \rightarrow 3 \rightarrow 4 \rightarrow 5$), safely arriving at $T = 4$.
2. **The Second Wave:** At $T = 2$, Node 1 processes a second wave of shares, routing 25 more down the direct cable to arrive precisely at the $T = 5$ deadline.

3. **Temporal Convergence and Collision Avoidance:** Node 2 processes exactly 40 shares and pushes them down its heavy cable ($2 \rightarrow 4$, latency 2). These arrive at Node 4 at exactly $T = 4$. The algorithm beautifully orchestrated the 15-share trickle from Node 1 to arrive at Node 4 early (at $T = 3$), ensuring Node 4's processing capacity of 40 was entirely free at $T = 4$ to handle Node 2's massive push. Those 40 shares hit the Sink precisely at $T = 5$.
4. **Final Tally:** The aggregated flow across these temporal waves is $25 + 15 + 25 + 40 = 105$ shares.

3.4 Stress Testing Edge Cases

For a comprehensive proof of validity, users should independently test the following extreme conditions:

- **Unreachable Deadlines:** Set $K = 2$ on the benchmark file. The output must strictly be 0, as no topological path to the Sink exists within 2 microseconds.
- **Zero Processing Capacity:** Set the default processing capacity to 0. The output must cleanly evaluate to 0 without throwing exceptions.
- **Disconnected Graphs:** Remove all edges connecting to the Sink. The residual graph must safely terminate and yield 0.